

Traffic Ramps and the Peeking Problem, Worked Out by Hand

Why a 5% canary bleeds 10 bad responses a day instead of 200 — and why checking your A/B five times turns a 5% false-alarm rate into 23%.

What this document does. It takes the staged rollout from Module 4.5 — the refund-agent ramp that goes *shadow* → *canary* → *partial* → *full* — and computes two things by hand. First, the **blast radius** at each stage: with a 2% defect rate over 10,000 requests/day, exactly how many bad responses reach users at 5%, 25%, 50%, and 100% live traffic. Second, the **peeking problem**: if you check the significance test m times and stop the moment you see $p < 0.05$, what your true false-positive rate becomes. Nothing is hand-waved; every number is reproduced by the Python program in Appendix A.

Where this sits. This is **Topic 2 of Module 4.5** (Versioning & A/B Testing) — the math behind the four rollout lanes and the “sequential peeking” caveat. Its companions: Topic 1 of this module (sample size and power — the test the peeking problem corrupts) and Module 4.2 (the dashboards and alerts that tempt you to peek).

Concepts covered, in order: staged rollout as a traffic fraction · risk exposure = fraction × defect rate × volume · exposure reduction from canarying · the family-wise error rate · why repeated peeking compounds $\alpha \cdot 1 - (1 - \alpha)^m$ · fixed-horizon vs. alpha-spending fixes · ramps as blast-radius control.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: a ramp is a dial on the blast radius

A staged rollout never flips a new agent version to all users at once. It turns a dial — the fraction of live traffic the candidate handles — up in steps, watching for trouble at each notch. The point of the dial is simple: if the candidate has a defect, you would rather discover it while it is touching 5% of users than 100%. To put numbers on that, we need three quantities.

Quantity	Symbol	Value here
Daily request volume	V	10,000 requests/day
Candidate defect rate	d	0.02 (2% of its responses are bad)
Traffic fraction on candidate	f	stage-dependent (0, .05, .25, .50, 1)
Significance level per look	α	0.05
Number of times we peek	m	varies (1 . . . 5)

Notation. *Risk exposure* E is the expected number of bad responses that actually reach users per day. The *family-wise error rate* (FWER) is the probability of *at least one* false positive across all the significance checks you make. The four rollout lanes from the module map to four values of f : shadow ($f = 0$, output discarded), canary ($f = 0.05$), partial ($f = 0.25$ then 0.50), and full ($f = 1.00$).

Intuition

The ramp and the peeking problem are two halves of the same trade. The ramp says “don’t expose everyone until you have evidence.” Peeking is the temptation to *rush* that evidence by checking the dashboard over and over and pouncing on the first green p-value. The first bounds your risk; the second, done naively, quietly *inflates* it. A disciplined rollout needs both the slow dial and an honest stop rule.

2 Step 1 — risk exposure at each stage

Only the fraction f of traffic on the candidate can be harmed by its defects, and of that fraction, a share d goes bad. So the expected bad responses per day is the product of all three quantities:

$$E(f) = f \cdot d \cdot V \quad (1)$$

It is linear in f : double the live fraction, double the daily harm. Working the five stages with $d = 0.02$ and $V = 10,000$:

Stage	Lane	Fraction f	Bad/day $E(f)$
Shadow	0% live	0.00	0
Canary	5% live	0.05	10
Ramp	25% live	0.25	50
Ramp	50% live	0.50	100
Full	100% live	1.00	200

Sample check, canary: $E = 0.05 \times 0.02 \times 10,000 = 0.05 \times 200 = 10$ bad responses/day. ✓ At full rollout the same defect produces $1.00 \times 200 = 200$ bad responses/day. The shadow stage is special: $f = 0$, so $E = 0$ — the candidate runs on a copy of real traffic but its output never reaches a user, which is exactly why the module insists shadow runs use read-only tool stubs and skip memory writes.

Mini-example — this operation in isolation

The whole value of canarying is one ratio. Compare the canary stage to full rollout:

$$\frac{E(\text{full})}{E(\text{canary})} = \frac{200}{10} = 20.$$

The canary exposes 20× fewer users to a bad version than a blind full rollout would — a 95% reduction in blast radius ($1 - 10/200 = 0.95$). And because d and V cancel in the ratio, that 20× holds *whatever* the defect rate or volume: the protection factor is just $f_{\text{full}}/f_{\text{canary}} = 1.00/0.05$. The dial setting alone determines how much you are shielded.

3 Step 2 — exposure reduction across the whole ramp

The point of staging is that you spend most of the dangerous early period at a *low* f , only widening once the evidence is in. Suppose a defect exists and takes one day at each of canary,

25%, and 50% before you catch it at the 50% gate and revert. The bad responses that escaped total

$$10 + 50 + 100 = 160 \text{ over three days,}$$

versus a blind full launch that would have produced 200 *per day* — 600 over the same three days. The ramp cut the realised harm from 600 to 160, a 73% reduction, *and* gave three independent days of dashboard signal before anyone bet the whole user base on the change.

Intuition

A ramp converts a single all-or-nothing gamble into a sequence of cheap, reversible bets. Each notch buys information at a bounded cost. The shadow stage buys the most information for the least risk ($E = 0$) but only catches defects visible offline; the canary catches the defects that only appear against live tool state, at 1/20 the exposure of a full launch. You climb the dial only as fast as your evidence accumulates — which is precisely where the temptation to peek comes from.

4 Step 3 — the peeking problem: why repeated looks over-reject

Here is the trap. You are running the A/B from Topic 1. The dashboard updates daily, and each day you run the significance test at $\alpha = 0.05$ and decide “ship” the moment you see $p < 0.05$. That feels harmless — each individual test has only a 5% false-positive rate. But you are not running *one* test; you are running one *per look*, and stopping at the first success. The chance of *at least one* false alarm across m independent looks is the complement of never alarming:

$$\text{FWER}(m) = 1 - (1 - \alpha)^m \quad (2)$$

This treats the looks as independent, which is an approximation — sequential looks on accumulating data are positively correlated, so the true inflation is somewhat *less* than this. But the approximation captures the essential disease: each extra look is another chance to be fooled, and the chances compound. Working it out:

Looks m	FWER = $1 - (0.95)^m$	as %
1	0.0500	5.0%
2	0.0975	9.8%
3	0.1426	14.3%
5	0.2262	22.6%
10	0.4013	40.1%
20	0.6415	64.2%

Sample check, $m = 5$: $0.95^5 = 0.7738$, so $\text{FWER} = 1 - 0.7738 = 0.2262$. ✓ Peeking five times turns the nominal 5% false-positive rate into **22.6%** — more than four times what you signed up for. Peek every day for three weeks ($m = 20$) and it is nearly two-in-three: you are almost *guaranteed* to “find” a winner that isn’t one.

Watch out

This is why naive sequential A/B testing over-rejects — it ships ties as wins. The mechanism

is subtle because nothing in any single day’s test is wrong; the error is in the *stopping rule*. “Run until significant, then stop” is a recipe for finding significance in pure noise, given enough looks. The longer an inconclusive experiment runs under daily peeking, the *more* likely you are to falsely call it — the opposite of the reassurance more data is supposed to give.

5 Step 4 — two honest fixes

You do not have to choose between “never look” and “get fooled.” Two standard remedies keep the overall false-positive rate at α :

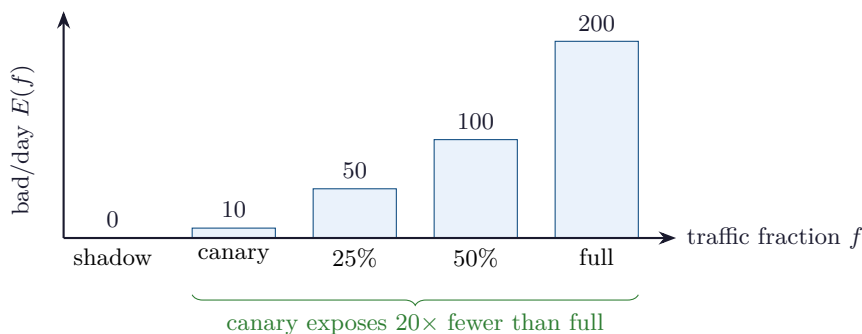
- **Fixed-horizon test.** Pre-register the sample size from Topic 1 ($N = 1247/\text{arm}$ for a 5-point lift), collect it *in full*, then test *once*. One look means $m = 1$, so $\text{FWER} = \alpha = 0.05$ exactly. The cost is discipline: you must not pounce early.
- **Alpha spending.** If you genuinely need to monitor as data arrives, spend your error budget across the looks instead of re-spending the full α each time. The blunt version is a Bonferroni split: to hold 0.05 over $m = 5$ looks, test each look at $0.05/5 = 0.01$. The exact (Šidák) per-look level is $1 - (1 - 0.05)^{1/5} = 0.0102$. Sequential methods like mSPRT do this adaptively, spending less early and more late.

Sample check, Bonferroni at $m = 5$: a per-look level of 0.01 gives a family-wise bound of $1 - (1 - 0.01)^5 = 1 - 0.99^5 = 1 - 0.9510 = 0.0490 \leq 0.05$. ✓ The correction works: by tightening each look you buy back the inflation, at the price of needing a slightly larger sample to keep the same power.

Intuition

Significance is a fixed budget, not a renewable one. Each peek spends a slice of your 5% error allowance; the fixes simply make you *account* for what you spend. Pre-register and look once (spend it all at the end), or look often but spend a sliver at a time. What you cannot do is spend the full 5% on every look and pretend the total is still 5% — that is the arithmetic of Equation (2), and it does not forgive.

6 The ramp and the looks, drawn



The bar chart *is* Equation (1): exposure climbs linearly with the dial, from 0 at shadow to 200 at full. The whole discipline of the ramp is to linger at the short bars — where a defect costs

10/day, not 200 — until the evidence (collected under an *honest* stop rule, not naive peeking) clears the gate to widen.

7 Ramp & peeking cheat-sheet

Risk exposure per day	$E(f) = f \cdot d \cdot V$
Canary vs. full protection	$f_{\text{full}}/f_{\text{canary}} = 1.00/0.05 = 20\times$
Blast-radius reduction (canary)	$1 - f_{\text{canary}}/f_{\text{full}} = 95\%$
Family-wise error of m looks	$\text{FWER}(m) = 1 - (1 - \alpha)^m$
Five looks at $\alpha=.05$	$1 - 0.95^5 = 0.2262$
Fixed-horizon fix	one look at the pre-registered N : $\text{FWER} = \alpha$
Bonferroni per-look level	α/m (e.g. $0.05/5 = 0.01$)
Šidák per-look level	$1 - (1 - \alpha)^{1/m}$ (e.g. 0.0102)

8 An honest word about these numbers

The defect rate ($d = 0.02$) and volume ($V = 10,000/\text{day}$) are illustrative round numbers, chosen so the exposure arithmetic is followable by hand; your real d comes from the candidate's eval, and V from your traffic. The FWER formula $1 - (1 - \alpha)^m$ assumes the m looks are *independent*, which they are not — sequential looks on growing data are correlated, so the true inflation is a bit *milder* than the table shows. That makes the formula a clean upper-bound-style intuition rather than an exact sequential calculation; the genuine fix (mSPRT, group-sequential boundaries) is more careful still. What is *not* illustrative is the structure: exposure is strictly *linear* in the live fraction, so a ramp buys a proportional cut in blast radius; and naive repeated peeking strictly *inflates* the false-positive rate, so an honest stop rule is mandatory. Change the constants and the figures move; both shapes — linear exposure, compounding error — do not.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the exposure table, the FWER table, and the correction checks; change `d`, `V`, `alpha`, or the stage fractions to model your own ramp.

```

1  # ramp_and_peeking.py -- reproduces every number in
2  # "Traffic Ramps and the Peeking Problem".
3
4  d = 0.02 # candidate defect rate (2% of responses bad)
5  V = 10_000 # daily request volume
6  alpha = 0.05 # significance level per look
7
8  def exposure(f): # Eq. (1): bad responses/day at live fraction f
9      return f * d * V
10
11 # --- exposure at each rollout stage ---
12 print("--- exposure table ---")
13 stages = [("shadow", 0.00), ("canary", 0.05),
14           ("ramp25", 0.25), ("ramp50", 0.50), ("full", 1.00)]
15 for name, f in stages:

```

```

16     print(f"{name:7s} f={f:.2f} bad/day={exposure(f):.0f}")
17
18 canary, full = exposure(0.05), exposure(1.00)
19 print(f"protection factor full/canary = {full/canary:.0f}x")
20 print(f"blast-radius reduction = {(1-canary/full)*100:.0f}%")
21
22 # --- peeking: family-wise error rate of m looks ---
23 def fwer(m): # Eq. (2): 1 - (1-alpha)^m
24     return 1 - (1 - alpha)**m
25
26 print("--- FWER table ---")
27 for m in (1, 2, 3, 5, 10, 20):
28     print(f"m={m:2d} FWER={fwer(m):.4f} ({fwer(m)*100:.1f}%)")
29
30 # --- corrections that restore FWER <= 0.05 over m=5 looks ---
31 m = 5
32 bonf = alpha / m # Bonferroni per-look level
33 sidak = 1 - (1 - alpha)**(1/m) # exact Sidak per-look level
34 print("--- corrections (m=5) ---")
35 print(f"Bonferroni per-look alpha = {bonf:.4f}")
36 print(f"Sidak per-look alpha = {sidak:.4f}")
37 print(f"FWER with Bonferroni = {1-(1-bonf)**m:.4f}")
38
39 # Expected output:
40 # --- exposure table ---
41 # shadow f=0.00 bad/day=0
42 # canary f=0.05 bad/day=10
43 # ramp25 f=0.25 bad/day=50
44 # ramp50 f=0.50 bad/day=100
45 # full f=1.00 bad/day=200
46 # protection factor full/canary = 20x
47 # blast-radius reduction = 95%
48 # --- FWER table ---
49 # m= 1 FWER=0.0500 (5.0%)
50 # m= 2 FWER=0.0975 (9.8%)
51 # m= 3 FWER=0.1426 (14.3%)
52 # m= 5 FWER=0.2262 (22.6%)
53 # m=10 FWER=0.4013 (40.1%)
54 # m=20 FWER=0.6415 (64.2%)
55 # --- corrections (m=5) ---
56 # Bonferroni per-look alpha = 0.0100
57 # Sidak per-look alpha = 0.0102
58 # FWER with Bonferroni = 0.0490

```

— end of document —